



*Akademia Górniczo-Hutnicza
w Krakowie*

Faculty: AGH	Academic Year 2025/2026	Year of study	Fields of science:
Student(s) name: Simon GANIER-LOMBARD & Daniil STEPANOV			
Course: Real Time Operating Systems in Control Applications			
Exercise nr: RTOS-06			
Exercise subject: Processes priorities and scheduling algorithms			
Lecturer: dr inż. Grzegorz Hayduk			
Date: 02.06.2026			Grade:

1 Exercise purpose

The goal of this exercise is to learn how priorities and scheduling policies work in QNX.

We create child processes with fork, and after give them different priorities, scheduling policies, and make each one run a heavy computation with no blocking calls.

Then we then look at the order in which they finish to see the effect of the priority and of the policy on this processes.

2 Assumptions / Theory

In QNX every process has a priority from 1 to 255, where 255 is the highest priority. The scheduler always runs the ready process with the highest priority, and a higher priority process can preempt a lower one.

There are three scheduling policies type:

- SCHED_FIFO: the process runs until the CPU it's by itself or a higher priority process preempts it.
- SCHED_RR: same as FIFO, but the process also gives up the CPU when it uses its time quantum.
- SCHED_SPORADIC: the priority drops by 1 when the budget runs out and is restored after the process blocks.

We set the priority with `sched_setparam` and the policy with `sched_setscheduler`, and we read them back with `sched_getparam` and `sched_getscheduler`.

The fork call makes a child process that is a copy of the parent; it returns 0 in the child and the child pid in the parent.

3 Description of implementation

We wrote two programs.

Both use fork to create children, shared memory for the results, and a semaphore so all children start at the same moment.

- ex6_1.c covers points 1 and 2.

It creates 10 children with priorities from 10 to 100. The parent opens two shared memory areas: one for the results and one for a semaphore gate.

Each child maps the same memory, sets its priority, prints READY, posts a ready semaphore, then waits on the gate semaphore.

The parent runs at a priority above every child, so it can post the gate for all of them before any child runs, then it blocks in waitpid.

Each child measures its own run time with *gettimeofday* and records the order in which it finishes with an atomic counter. The parent prints the results in finish order at the end.

- ex6_2.c covers points 3 and 4.

It creates 10 children spread across the three policies (SCHED_FIFO, SCHED_RR, SCHED_SPORADIC), all with the same priority 20, so the policy is the only thing that differs.

It uses two semaphores: a ready semaphore so the parent knows every child is set up, and a gate semaphore to release them all at once.

Each child records its start rank and finish rank with an atomic counter, so we can see the order in which they start and finish.

Each child also reads back its real policy with *sched_getscheduler*.

If *sched_setscheduler* fails, the child still forces its priority with *sched_setparam*, so every child keeps the same priority and the comparison stays fair.

The computation in both programs is a plain loop with no blocking calls. The elapsed time of one child has little meaning, because every child runs the same loop, so the meaningful result is the order in which the children finish.

See the full commented code in the .zip file.

```
Console X Debugger Console Memory Executab
<terminated> ex6-1 [C/C++ QNX Application] /tmp/ex6-1 on C
READY priority=10 pid=2002970
READY priority=20 pid=2002971
READY priority=30 pid=2002972
READY priority=40 pid=2002973
READY priority=50 pid=2002974
READY priority=60 pid=2002975
READY priority=70 pid=2002976
READY priority=80 pid=2002977
READY priority=90 pid=2002978
READY priority=100 pid=2002979

All children ready, releasing them together...

DONE priority=100 finish=1 2.065127 s
DONE priority=90 finish=2 2.125441 s
DONE priority=80 finish=3 2.085288 s
DONE priority=70 finish=4 2.099543 s
DONE priority=60 finish=5 2.095401 s
DONE priority=50 finish=6 2.088389 s
DONE priority=40 finish=7 2.251435 s
DONE priority=30 finish=8 2.304412 s
DONE priority=20 finish=9 2.152660 s
DONE priority=10 finish=10 2.097879 s

=== Results (finish order) ===
finish 1: priority 100 2.065127 s
finish 2: priority 90 2.125441 s
finish 3: priority 80 2.085288 s
finish 4: priority 70 2.099543 s
finish 5: priority 60 2.095401 s
finish 6: priority 50 2.088389 s
finish 7: priority 40 2.251435 s
finish 8: priority 30 2.304412 s
finish 9: priority 20 2.152660 s
finish 10: priority 10 2.097879 s
```

Console output of ex6_1.c: 10 children, priorities 10 to 100, results in finish order.

```

Waiting for all 10 children to be ready...
Launching all childs
DONE [SCHED_SPORADIC :0] pid=1847324 start=3 finish=2 0.025136 s
DONE [SCHED_SPORADIC :1] pid=1847327 start=6 finish=4 0.024483 s
DONE [SCHED_SPORADIC :2] pid=1847330 start=9 finish=6 0.024650 s
DONE [SCHED_FIFO :0] pid=1847322 start=1 finish=1 0.024223 s
DONE [SCHED_FIFO :1] pid=1847325 start=4 finish=3 0.024297 s
DONE [SCHED_FIFO :2] pid=1847328 start=7 finish=5 0.024253 s
DONE [SCHED_FIFO :3] pid=1847331 start=10 finish=7 0.024097 s
DONE [SCHED_RR :1] pid=1847326 start=5 finish=9 0.092212 s
DONE [SCHED_RR :0] pid=1847323 start=2 finish=8 0.116366 s
DONE [SCHED_RR :2] pid=1847329 start=8 finish=10 0.064347 s

```

Console output of ex6_2.c: 10 children at priority 20 across the three policies.

4 Conclusions

This exercise showed us how the QNX scheduler uses priority and policy to choose which process runs. In the first program the children finished in priority order, the priority 100 child first and the priority 10 child last, which matches the preemptive rule. The order is what matters here, not the elapsed time, because every child runs the same loop. In the second program all ten children had the same priority 20, so the policy decided the order. The FIFO children each ran to completion and finished fast at about 0.024 s, while the RR children shared the CPU in time slices and took much longer, from 0.064 s to 0.116 s. The tricky part was to make the comparison fair. The parent must run at a priority above every child, so it can release all children before any of them runs. Without this the first released child preempts the parent and runs to completion, and a low priority child can wait on the semaphore forever. We also used shared memory, so the children can write their results back to the parent. One point to remember is to always check the return value of the sched_ functions, because if they return -1 the priority or policy did not change.

SCHED_SPORADIC is not implemented on the lab QNX version, so the sched_setscheduler calls for it failed with Invalid argument. Those children kept the inherited FIFO policy and behaved like the FIFO group.